# 1.INTRODUCTION

## 1.1 Background and Motivation

Pregnancy and maternity care require continuous medical attention, timely monitoring, and accurate record management to ensure the health and safety of both mother and child. Traditionally, maternity care records such as appointments, test results, prescriptions, and trimester updates are maintained manually or through fragmented digital systems. This often leads to problems like data loss, delayed communication, difficulty in tracking patient history, and lack of coordination between hospitals, doctors, and patients.

With the rapid growth of healthcare services and increasing number of patients, hospitals and maternity clinics face challenges in efficiently managing patient information and appointments. Expectant mothers also need easy access to their medical reports, doctor advice, and appointment status without repeatedly visiting hospitals. Hence, there is a strong need for a centralized, secure, and user-friendly maternity care management system.

Bloom Moms Care is designed to address these challenges by providing a digital platform that integrates patient management, doctor interaction, appointment scheduling, and medical record handling into a single system**.**

The motivation behind developing Bloom Moms Care arises from the need to improve the quality and accessibility of maternity healthcare through technology. Pregnant women require special care, regular follow-ups, and clear communication with healthcare providers. Delays or mismanagement during this critical period can lead to serious health risks.

By implementing a web-based maternity care system using Python Django and MySQL, Bloom Moms Care aims to enhance healthcare efficiency, improve doctor-patient communication, and ensure better maternity care outcomes.

## 1.2 The Proposed System

**Bloom Moms Care,** is a web-based maternity care management system designed to digitally manage and monitor maternity-related healthcare activities in an efficient, secure, and organized manner. The system aims to replace traditional

manual record-keeping with a centralized digital platform that enhances coordination between administrators, doctors, and patients.

The system follows a role-based access approach with three main stakeholders: Admin, Doctor, and Patient. Each user is provided with specific functionalities based on their role to ensure data security and proper workflow management.

**Admin** acts as the central authority of the system. The admin is responsible for managing doctors and patients, approving and rescheduling appointments, assigning doctors to patients, and uploading hospital test results. The admin can monitor trimester updates and patient progress throughout pregnancy. This ensures smooth system operation and accurate maintenance of maternity records.

**Doctors** can view only the patients assigned to them by the admin. They can upload prescriptions, medical notes, trimester updates, and other medical records. Doctors do not have permission to approve or reschedule appointments, ensuring controlled access and preventing data misuse. This module helps doctors track patient health history efficiently and provide timely medical care.

**Patients** can log in using credentials provided by the admin. They can view doctor details, book appointments, check appointment status (accepted or rescheduled), upload external medical test results, and view prescriptions, trimester updates, and medical history. After six months of delivery, the patient profile is automatically locked, allowing only appointment booking to ensure data integrity and privacy.

The proposed system will have the following key features:

- Centralized database using **MySQL** for secure data storage
- Web-based interface developed using **Python Django**
- Role-based authentication and authorization
- Secure handling of medical records and reports
- Efficient appointment management and tracking
- Improved doctor–patient communication

## 1.3 Project Scope

### Limitations of the Existing System

The existing maternity care management systems in many hospitals and clinics are largely manual or partially computerized. These systems have several limitations that affect the efficiency and quality of maternity healthcare services.

1. **Manual Record Maintenance**
   Most maternity records such as patient details, appointment schedules, test reports, and prescriptions are maintained on paper. This increases the risk of data loss, damage, and misplacement.
2. **Time-Consuming Processes**
   Booking appointments, searching patient history, and updating medical records require more time in manual systems, leading to delays in treatment and increased workload for hospital staff.
3. **Lack of Centralized Data**
   Patient information is often stored in different registers or systems, making it difficult to access complete medical history when required.
4. **Poor Doctor–Patient Communication**
   Patients do not receive timely updates about appointment status, test results, or medical advice, which can affect proper maternity care.
5. **Limited Accessibility**
   Patients must visit hospitals frequently to check appointment status or collect reports, which is inconvenient, especially during pregnancy.
6. **Higher Chances of Errors**
   Manual data entry increases the possibility of human errors, such as incorrect patient details, wrong dates, or missing records.
7. **Data Security Issues**
   Existing systems lack proper security mechanisms, making sensitive medical data vulnerable to unauthorized access.

8. **Difficulty in Tracking Pregnancy Progress**
   Monitoring trimester-wise updates and long-term patient history is difficult without a dedicated digital system.

**Advantages of the Proposed System**

1. **Centralized Data Management**
   All maternity-related information such as patient details, appointments, test results, prescriptions, and trimester updates are stored in a single centralized database, making data access easy and efficient.
2. **Reduced Paperwork**
   The system eliminates manual record keeping, thereby reducing paperwork, storage issues, and the risk of data loss or damage.
3. **Improved Efficiency and Time Saving**
   Appointment booking, rescheduling, and record updates are performed digitally, saving time for administrators, doctors, and patients.
4. **Role-Based Access Control**
   Separate access for Admin, Doctor, and Patient ensures secure handling of sensitive medical data and prevents unauthorized access.
5. **Better Doctor–Patient Communication**
   Patients can easily view prescriptions, medical notes, and trimester updates, ensuring clear communication and better maternity care.
6. **Accurate and Reliable Records**
   Digital storage minimizes human errors and ensures accurate, consistent, and up-to-date medical records.
7. **Easy Accessibility**
   Patients can access their medical information anytime through the web-based system without frequent hospital visits.
8. **Efficient Appointment Management**
   Admin-controlled appointment approval and rescheduling avoid conflicts and improve hospital workflow.

9. **Secure Medical Data**
   The system uses authentication and database security to protect sensitive patient information.

10. **Scalability and Future Enhancement**
    The system can be extended in the future to include features like email notifications, reports, and advanced analytics.

# 2. SYSTEM ANALYSIS

## 2.1 INTRODUCTION

Software Engineering is the analysis, design, construction, verification and management of technical or social entities. To engineer software accurately, a software engineering process must be defined. System analysis is a detailed study of the various operations performed by the system and their relationship within and module of the system. It is a structured method for solving the problems related to the development of a new system. The detailed investigation of the present system is the focal point of system analysis. This phase involves the study of parent system and identification of system objectives. Information has to be collected from all people who are affected by or who use the system. During analysis, data are collected on the variable files, decision point and transactions handled by the present system. The main aim of system is to provide the efficient and user-friendly automation. So, the system analysis process should be performed with extreme precision, so that an accurate picture of existing system, its disadvantages and the requirements of the new system can be obtained.

System analysis involves gathering the necessary information and using the structured tool for analysis. This includes the studying existing system and its drawback, designing a new system and conducting cost benefit analysis. System analysis is a problem-solving activity that requires intensive communication between the system users and system developers. The system is studied to the minute detail and analyzed. The system is viewed as a whole and the inputs to the system are identified. The outputs from the organization are traced through various phases of processing of inputs. There are a number of different approaches to system analysis. When a computer-based information system is developed, systems analysis (according to the Waterfall model) would constitute the following steps:

• The development of a feasibility study, involving determining whether a project is economically, technologically and operationally feasible.

• Conducting fact-finding measures, designed to ascertain the requirements of the system's end-users. These typically span interviews, questionnaires, or visual observations of work on the existing system.

• Gauging how the end-users would operate the system (in terms of general experience in using computer hardware or software), what the system would be used for and so on Techniques such as interviews, questionnaires etc. can be used for the detailed study of these processes. The data collected by these sources must be scrutinized to arrive at a conclusion.

The conclusion is an understanding of how the system functions. This system is called the Existing System. The Existing system is then subjected to close observation and the problem areas are identified. The designer now functions as a problem solver and tries to sort out the difficulties that the enterprise faces. The solutions are given as a proposal which is the Proposed System. The proposal is then weighed with the existing system analytically and the best one is selected. The proposal is then presented to the user for an endorsement by the user. The proposal is reviewed on user request and suitable changes are made. This is a loop that ends as soon as the user is satisfied with the proposal.

## 2.2 STAKEHOLDERS IN THIS PROJECT

### 2.2.1 ADMIN

The Admin is the primary stakeholder responsible for managing the overall system. The admin handles doctor and patient management, appointment approval and rescheduling, doctor assignment, uploading hospital test results, and monitoring trimester updates. The admin ensures smooth system operation and data accuracy.

Responsibilities and Interactions:

- Manage doctor and patient accounts
- Approve and reschedule patient appointments
- Assign doctors to patients
- Upload hospital test results and reports

- Monitor trimester updates and patient progress
- Maintain overall system security and data accuracy
- Interacts with Doctors by assigning patients and providing access to patient records
- Interacts with Patients by managing appointment status and ensuring proper service flow
- Acts as the central controller between doctors and patients

### 2.2.2 DOCTOR

Doctors are responsible for providing medical care to patients using the system. They can view assigned patients, access patient medical history, upload prescriptions, medical notes, trimester updates, and medical records. Doctors use the system to efficiently monitor patient health and provide timely treatment.

- View assigned patient details and medical history
- Upload prescriptions and medical notes
- Update trimester-wise health information
- Upload medical records related to maternity care
- Provide timely and accurate medical guidance
- Interacts with Admin to receive assigned patients
- Interacts with Patients by updating prescriptions and trimester information
- Does not interact directly with appointment approval or rescheduling

### 2.2.3 PATIENT

Patients are the end users of the system. They can view doctor details, book appointments, check appointment status, upload external test results, and view prescriptions, trimester updates, and medical history. The system helps patients stay informed and engaged in their maternity care.

- Book appointments with doctors
- Upload external test reports and medical documents
- Follow prescribed treatment and medical advice
- Regularly check appointment status and health updates
- Interacts with Admin for appointment approval and system access

- Interacts with Doctors by viewing prescriptions and trimester updates
- Accesses medical records securely through the system

## 2.3 SOFTWARE REQUIREMENT SPECIFICATION

**Features Provided to Admin**

1. The system should allow the Admin to log in securely using credentials.

2. The system should allow the Admin to add new doctors into the system.

3. The system should allow the Admin to edit or update doctor details.

4. The system should allow the Admin to activate or deactivate (block/unblock) a doctor.

5. The system should allow the Admin to view the list of all registered doctors.

6. The system should allow the Admin to add new patients into the system.

7. The system should allow the Admin to edit or update patient details.

8. The system should allow the Admin to activate or deactivate a patient.

9. The system should allow the Admin to assign a maternity doctor to a patient.

10. The system should allow the Admin to manage appointment requests made by patients.

11. The system should allow the Admin to accept or reschedule appointments (no rejection).

12. The system should allow the Admin to upload hospital-based test results to patient medical records.

13. The system should allow the Admin to monitor trimester updates uploaded by doctors.

14. The system should automatically lock the patient maternity profile after 6 months of delivery.

15. The system should allow the Admin to manage general (non-maternity) appointments after profile lock.

16. The system should allow the Admin to view statistics such as total doctors, patients, appointments, and locked maternity profiles.

17. The system should allow the Admin to ensure smooth hospital operation and maintain maternity and postpartum records.

**Features Provided to Doctor**

1. The system should allow the Doctor to log in using valid credentials.

2. The system should allow the Doctor to view all currently assigned maternity patients.

3. The system should allow the Doctor to view patient details and medical history.

4. The system should allow the Doctor to provide trimester-wise guidance (1st, 2nd, 3rd).

5. The system should allow the Doctor to upload prescriptions after each visit. 6. The system should allow the Doctor to upload medical notes and advice for patients.

7. The system should allow the Doctor to view test results uploaded by Admin or Patient.

8. The system should allow the Doctor to track pregnancy progress (weeks, trimesters).

9. The system should allow the Doctor to view previously submitted trimester updates, prescriptions, and visit history.

10. The system should automatically remove the patient from the doctor's active maternity list after the 6-month postpartum period.

**Features Provided to Patient**

1. The system should allow the Patient to login using credentials provided by the Admin.

2. The system should allow the Patient to view their profile details added by the Admin.

3. The system should allow the Patient to update limited profile information (address, phone, password).

4. The system should allow the Patient to view details of their assigned maternity doctor.

5. The system should allow the Patient to book maternity appointments with their assigned doctor.

6. The system should allow the Patient to view appointment status (Pending / Accepted / Rescheduled).

7. The system should allow the Patient to upload external test results or scan reports.

8. The system should allow the Patient to receive trimester updates provided by the Doctor.

9. The system should allow the Patient to access and download prescriptions and medical notes.

10. The system should allow the Patient to view pregnancy progress, visit history, and medical records.

11. The system should automatically restrict maternity-access features after 6 months postdelivery.

12. The system should allow the Patient (after profile lock) to book general gynecology appointments from the available doctor list.

| **Sl.No** | **Name & Designation** | **Date** | **Accepted (Yes or No)** |
|---|---|---|---|
| 1 | Mr.Aswin T.S<br>Assistant Professor<br>Santhigiri College of Computer Sciences | | |
| 2 | Merin P Eldho<br>Developer | | |

## 2.4 FEASIBILITY STUDY

A feasibility study is conducted to evaluate the practicality and viability of the proposed system before development. The Bloom Moms Care project has been analyzed based on technical, economic, operational, and schedule feasibility.

**Technical Feasibility**

The proposed system is technically feasible as it uses well-established and widely supported technologies such as **Python Django** for backend development and **MySQL** for database management. These technologies are compatible with standard hardware and operating systems. The development team has sufficient knowledge of web technologies, making implementation and maintenance feasible. The system can be easily hosted on common web servers and supports future enhancements.

**Economic Feasibility**

The project is economically feasible because it is developed using **open-source technologies**, which eliminates licensing costs. The system requires minimal hardware resources and can run on existing infrastructure. By reducing paperwork, manual labor, and time spent on administrative tasks, the system helps in cost savings for hospitals and clinics.

**Operational Feasibility**

The system is operationally feasible as it is user-friendly and designed to meet the needs of all stakeholders—Admin, Doctor, and Patient. Role-based access ensures ease of use and security. The system simplifies appointment management, record maintenance, and communication, making it easy to adopt with minimal training.

## 2.5 SOFTWARE DEVELOPMENT LIFE CYCLE MODEL

One of the basic notions of the software development process is SDLC models which stand for Software Development Life Cycle models. SDLC – is a continuous process, which starts from the moment, when it’s made a decision to

launch the project, and it ends at the moment of its full remove from the exploitation. Software development lifecycle (SDLC) is a framework that defines the steps involved in the development of software. It covers the detailed plan for building, deploying and maintaining the software. SDLC defines the complete cycle of development i.e. all the tasks involved in gathering a requirement for the maintenance of a product. Some of the common SDLC models are Waterfall Model, V-Shaped Model, Prototype Model, Spiral Model, Iterative Incremental Model, Big Bang Model, Agile Model. We used Agile Model for our Project.

Agile Model Agile Model is a combination of Iterative and incremental model. This model focuses more on flexibility while developing a product rather than on the requirement. In the agile methodology after every development iteration, the client is able to see the result and understand if he is satisfied with it or he is not. Extreme programming is one of the practical uses of this model. The basis of this model consists of short meetings where we can review our project. In Agile, a product is broken into small incremental builds. It is not developed as a complete product in one go. At the end of each sprint, the project guide verifies the product and after his approval, it is finalized. Client feedback is taken for improvement and his suggestions and enhancement are worked on in the next sprint. Testing is done in each sprint to minimize the risk of any failures. Advantages of Agile Model:

• It allows more flexibility to adapt to the changes.

• The new feature can be added easily.

• Customer satisfaction as the feedback and suggestions are taken at every stage.
• Risks are minimized thanks to the flexible change process.

**Disadvantages of Agile Model:**

- Lack of documentation.
- If a customer is not clear about how exactly they want the product to be, then the project would fail.
- With all the corrections and changes there is possibility that the project will exceed expected time.

## 2.6 HARDWARE AND SOFTWARE REQUIREMENTS

### 2.6.1 SOFTWARE SPECIFICATION

This project is built upon the latest technology software.

Front end : HTML, CSS, JavaScript

Development tool : Python Django

Database : MySQL

Web server : WAMP server

Operating System : Windows 11

#### 2.6.1.1 Python Django

Django is a high-level, open-source web framework written in Python that enables rapid development of secure and scalable web applications. It follows the Model–View–Template (MVT) architectural pattern, which helps in organizing application code efficiently.

Django provides built-in features such as user authentication, URL routing, form handling, session management, and an Object Relational Mapper (ORM). The ORM allows developers to interact with the database using Python code instead of writing complex SQL queries, thereby reducing development time and errors.

One of the key strengths of Django is its strong focus on security. It protects applications against common web vulnerabilities such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF). This makes Django highly suitable for healthcare systems where data confidentiality and integrity are critical.

Django also supports rapid development through its reusable components, admin interface, and modular design. Its scalability and flexibility allow the system to be enhanced in the future with additional features such as notifications, reporting, and analytics.

In the Bloom Moms Care project, Django is used to manage user roles, handle appointment workflows, process medical records, and ensure secure interaction between Admin, Doctor, and Patient modules.

#### 2.6.1.2 MySQL

**MySQL** is an open-source **relational database management system (RDBMS)** that is used to store, organize, and manage data efficiently. It uses **Structured Query Language (SQL)** to perform database operations such as storing, retrieving, updating, and deleting data.

MySQL stores data in the form of tables consisting of rows and columns, which helps in maintaining data consistency and integrity. It supports relationships between tables using primary keys and foreign keys, making it suitable for handling complex data such as patient records, appointments, prescriptions, and medical reports.

MySQL provides high performance, reliability, and scalability, making it widely used in web-based applications. It also supports data security features such as user authentication and access control, which are essential for protecting sensitive medical information.

In the **Bloom Moms Care** project, MySQL is used as the backend database to store details related to Admin, Doctors, Patients, Appointments, Test Results, Prescriptions, and Trimester Updates. Django's ORM is used to interact with the MySQL database efficiently, reducing the need for complex SQL queries.

#### 2.6.1.3 WAMP SERVER

WAMP Server is a Windows web development environment. It allows you to create web applications with Apache2, PHP and a MySQL database. Alongside, PhpMyAdmin allows you to manage easily your databases. WAMP Server refers to a software stack for the Microsoft Windows operating system, created by Romain Bourdon and consisting of the Apache web server, Open SSL for SSL support, MySQL database and PHP programming language. WAMP Server is a Web development platform on Windows that allows you to create dynamic Web applications with Apache2, PHP, MySQL and MariaDB. WampServer automatically installs everything you need to intuitively develop Web applications. You will be able to tune your server without even touching its setting files. Best of all, WampServer is available for free (under GPML license) in both 32 and 64-bit versions. Wamp server is not compatible with Windows XP, SP3, or Windows Server 2003. WAMP Server's functionalities are very complete and easy to use so we won't explain here how to use them. With a left click on WAMP Server's icon, you will be able to:

• switch online/offline (give access to everyone or only localhost)

• install and switch Apache, MySQL and PHP releases

• manage your server's settings

• access your logs

• access your settings files

• create alias

### 2.6.1.4 VISUAL STUDIO

Visual Studio Code is a source-code editor that can be used with a variety of programming languages, including C, C#, C++, FORTRAN, Go, Java, JavaScript, Node.js, Python, Rust, and Julia. It is based on the Electron framework, which is used to develop Node.js web applications that run on the Blink layout engine. Visual Studio Code employs the same editor component (code named "Monaco") used in Azure DevOps. Out of the box, Visual Studio Code includes basic support for most common programming languages. This basic support includes syntax highlighting, bracket matching, code folding, and configurable snippets. Visual Studio Code also ships with Intelligent for JavaScript, TypeScript, JSON, CSS, and HTML. Support for additional languages can be provided by freely available extensions on the VS Code Marketplace. Instead of a project system, it allows users to open one or more directories, which can then be saved in work spaces for future reuse. This allows it to operate as a language agnostic code editor for any language. It supports many programming languages and a set of features that differs per language. Unwanted files and folders can be excluded from the project tree via the settings. Many Visual Studio Code features are not exposed through menus or the user interface but can be accessed via the command palette.

### 2.6.1.5 WINDOWS 11

Operating System is defined as a program that manages the computer hardware. An operating system can be viewed as a scheduler, where it has resources for which it has charge. Resources include CPU, memory, I/O device and disk space. In another view, the operating system is a new machine. The third view is that operating system is a multiplexer which allows sharing of resources provides protection from interference and provides a level of cooperation between users.

This project is developed using Windows 11 as the operating system and supports its latest versions. Windows 11 is the latest installment in Microsoft's Windows NT family of operating systems, succeeding Windows 10. It was officially announced on June 24, 2021, and released to manufacturing on October 5, 2021, with general availability starting on October 5, 2021. Windows 11 introduces a refreshed user interface with centered Start Menu and taskbar icons, enhanced snap layouts, and virtual desktops. It also integrates Microsoft Teams directly into the operating system, providing seamless communication and collaboration capabilities. Other new features include support for running Android apps through the Amazon Appstore, a revamped Microsoft Store with a broader range of applications, improved gaming performance through Auto HDR and Direct Storage, and enhanced security features such as Windows Hello for faster, more secure login options. Initial reviews of Windows 11 have been generally positive, praising its design changes, performance improvements, and new features. Critics have noted concerns over hardware requirements and compatibility issues, particularly with older hardware, as well as some controversial decisions such as the requirement for a Microsoft account and internet connection during setup. Despite these criticisms, Windows 11 represents Microsoft's continued evolution of its operating system, aiming to enhance productivity, security, and user experience across a range of devices.

### 2.6.1.6 MICROSOFT WORD

Microsoft Word (or simply Word) is a word processor developed by Microsoft. It was first released on October 25, 1983 under the name Multi-Tool Word for Xenix systems. Subsequent versions were later written for several other platforms including IBM PCs running DOS (1983), Apple Macintosh running the Classic Mac OS (1985), AT&T Unix PC (1985), Atari ST (1988), OS/2 (1989), Microsoft Windows (1989), SCO Unix (1994), and mac OS (formerly OS X; 2001).

Commercial versions of Word are licensed as a standalone product or as a component of Microsoft Office, Windows RT or the discontinued Microsoft Works suite. Unlike most MSDOS programs at the time, Microsoft Word was designed to be used with a mouse. Advertisements depicted the Microsoft Mouse, and described Word as a WYSIWYG, windowed word processor with the ability to undo and display bold, italic, and underlined text, although it could not render fonts. It was not initially popular, since its user interface was different from the

leading word processor at the time, WordStar. However, Microsoft steadily improved the product, releasing versions 2.0 through 5.0 over the next six years. In 1985, Microsoft ported Word to the classic Mac OS (known as Macintosh System Software at the time). This was made easier by Word for DOS having been designed for use with highresolution displays and laser printers, even though none were yet available to the general public. Following the precedents of Lisa Write and MacWrite, Word for Mac OS added true WYSIWYG features. It fulfilled a need for a word processor that was more capable than MacWrite. After its release, Word for Mac OS's sales were higher than its MS-DOS counterpart for at least four years.

### 2.6.2 HARDWARE REQUIREMENTS

The selection of hardware configuring is a very task related to the software development, particularly inefficient RAM may affect adversely on the speed and corresponding on the efficiency of the entire system. The processor should be powerful to handle all the operations. The hard disk should have the sufficient to solve the database and the application.

Hardware used for development:

CPU : Intel i5 Processer

Memory : 4 GB

Cache : 6 MB

Hard Disk : 1 TB

Monitor : 15.6” Monitor

Keyboard : Standard108 keys Enhanced Keyboard

Mouse : Optical Mouse

Minimum Hardware Required for Implementation:

CPU : Pentium IV Processor

Memory : 256MB Above

Cache : 512 KB Above

Hard Disk : 20 GB Above

Monitor : Any

Keyboard : Any

Mouse : Any

# 3. SYSTEM DESIGN

## 3.1 SYSTEM ARCHITECTURE

A system architecture or system's architecture is the conceptual model that defines the structure, behavior, and more views of a system. An architecture description is a formal description and representation of a system, organized in a way that supports reasoning about the structures of the system. System architecture can comprise system components, the externally visible properties of those components, the relationships (e.g. the behavior) between them. It can provide a plan from which products can be procured, and systems developed, that will work together to implement the overall system. There have been efforts to formalize languages to describe system architecture; collectively these are called architecture description languages (ADLs). The system architecture can best be thought of as a set of representations of an existing (or to be created) system. It is used to convey the informational content of the elements comprising a system, the relationships among those elements, and the rules governing those relationships. The architectural components and set of relationships between these components that architecture describes may consist of hardware, software, documentation, facilities, manual procedures, or roles played by organizations or people. System architecture is primarily concerned with the internal interfaces among the system's components or subsystems, and the interface between the system and its external environment, especially the user.

The structural design reduces complexity, facilitates change and result in easier implementation by encouraging parallel development of different parts of the system. The procedural design transforms structural elements of program architecture into a procedural description of software components. The

architectural design considers architecture as the most important functional requirement. The system is based on the three-tier architecture.

The first level is the user interface (presentation logic), which displays controls, receives and validates user input. The second level is the business layer (business logic) where the application specific logic takes place. The third level is the data layer where the application information is stored in files or database. It contains logic about to retrieve and update data. The important feature about the three-tier design is that information only travels from one level to an adjacent level.

## 3.2 MODULE DESIGN

Modular programming is a software design technique that emphasizes separating the functionality of a program into independent, interchangeable modules, such that each contains everything necessary to execute only one aspect of the desired functionality. Conceptually, modules represent a separation of concerns, and improve maintainability by enforcing logical boundaries between components.

Different modules of this project include:

1.Authentication

2.Registration

3.Activities

4.Notifications

5. Report

**Authentication**

Authentication ensures secure access to the Bloom Moms Care system by verifying the identity of users. Each user—Admin, Doctor, and Patient—must log in using valid credentials such as username and password. Role-based authentication controls access to system features and protects sensitive medical data from unauthorized users.

**Registration**

Registration is the process of creating user accounts in the system. Doctors and patients are registered by the Admin to ensure data authenticity and security.

During registration, essential details such as personal information, contact details, and role-specific data are collected and stored securely in the database.

**Activities**

Activities refer to the various operations performed by users within the system. These include appointment booking and management, uploading medical reports, updating prescriptions, recording trimester updates, and viewing medical history. The system tracks these activities to ensure proper workflow and accountability.

**Notifications**

Notifications provide important updates to users regarding system events such as appointment approval or rescheduling, availability of prescriptions, and medical updates. Notifications help keep users informed in a timely manner and improve communication between Admin, Doctors, and Patients. *(Future enhancement: email notifications)*

**Report**

The reporting feature allows the Admin and Doctors to view and analyze patient-related information in an organized format. Reports include appointment details, medical history, test results, and trimester progress. These reports help in decision-making, monitoring patient health, and maintaining accurate medical records.

## 3.3 DATABASE DESIGN

A database is a collection of interrelated data stored with minimum redundancy to serve many users quickly and efficiently. The general objective is to make information access easy, quick, inexpensive and flexible for the users. The general theme behind a database is to integrate all information. Database design is recognized as a standard of management information system and is available virtually for every computer system. In database design several specific objectives are considered:

• Ease of learning and use

• Controlled redundancy

• Data independence

• More information at low cost

• Accuracy and integrity

• Recovery from failure

• Privacy and security

• Performance

A database is an integrated collection of data and provides centralized access to the data. Usually, the centralized data managing the software is called RDBMS. The main significant difference between RDBMS and other DBMS is the separation of data as seen by the program and data has in direct access to stores device. This is the difference between logical and physical data. 3.3.1 Normalization Designing a database is complete task and the normalization theory is a useful aid in the design process. The process of normalization is concerned with transformation of conceptual schema into computer representation form. There will be need for most databases to grow by adding new attributes and new relations. The data will be used in new ways. Tuples will be added and deleted. Information stored may undergo updating also. New association may also be added. In such situations the performance of a database is entirely depend upon its design. A bad database design may lead to certain undesirable things like:

• Repetition of information

• Inability to represent certain information

• Loss of information

To minimize these anomalies, Normalization may be used. If the database is in a normalized form, the data can be growing without, in most cases, forcing the rewriting application programs. This is important because of the excessive and growing cost of maintaining an organization's application programs and its data from the disrupting effects of database growth. As the quality of application programs increases, the cost of maintaining the without normalization will rise to prohibitive levels. A normalized database can also encompass many related activities of an organization thereby minimizing the need for rewriting the

applications of programs. Thus, normalization helps one attain a good database design and there by ensures continued efficiency of database.

Normalization theory is built around the concept of normal forms. A relation is said to be in normal form if it satisfies a certain specified set of constraints. For example, a relation is said to be in first normal form (1NF) if it satisfies the constraint that it contains atomic values only. Thus, every normalized relation is in 1NF.Numerous normal forms have been defined. Codd defined the first three normal forms. All normalized relations are in 1NF, some 1NF relations are also in 2NF and some 2NF relations are also in 3NF.2NF relations are more desirable than 1Nf and 3NF are more desirable than 2NF. That is, the database designer should prefer 3NF than 1NF or 2NF.Normalization procedure states that a relation that is in some given normal form can be converted into a set of relations in a more desirable form. We can define this procedure as the successive reduction of a given collection of relations to some more desirable form. This procedure is reversible. That is, it is always possible to take the output from the procedure and convert them back into input. In this process, no information is lost. So, it is also called “no loss decomposition”.

**First Normal Form**

A relation is in first normal form (1NF) if and all its attributes are based on single domain. The objective of normalizing a table is to remove its repeating groups and ensure that all entries of the resulting table have at most single value.

**Second Normal Form**

A table is said to be second Normal Form (2NF), when it is in 1NF and every attribute in record is functionally dependent upon the whole key, and not just a part of the key.

**Third Normal Form**

A table is in third Normal Form (3NF), when it is in 2NF and every non-key attribute is functionally dependent on just the primary key.

**3.3.2 Table Structure**

Table is a collection of complete details about a particular subject. These data are saved in rows and Columns. The data of each Row are different units. Hence, rows are called RECORDS and Columns of each row are called FIELDS. Data is

stored in tables, which is available in the backend the items and data, which are entered in the input, form id directly stored in this table using linking of database. We can link more than one table to input forms. We can collect the details from the different tables to display on the output.

There are  Tables in the project.They are,

1.tbl_login

2.tbl_patient

3.tbl_doctor

4.tbl_appointment

5.tbl_trimester_update

6.tbl_prescription

7.tbl_medical_record

8.tbl_visit_history

9.tbl_notifications

10.tbl_delivery_details

11.tbl_doctor_leave

1.Table **tbl_login**

Description : The table is used to store login details.

Table 3.1 tbl_login

| **Field Name** | **Data Type** | **Constraint** | **Description of Field** |
|---|---|---|---|
| Login_id | int | Primary Key | The field is used to uniquely identify logged in user,and the values are automatically generated. |

| | | | |
|---|---|---|---|
| User_name | Varchar(50) | Not null | This field stores the username. |
| password | Varchar(20) | Not null | This field stores the password. |
| role | Varchar(20) | Not null | This field stores the role. |
| status | Varchar(20) | Not null | This field stores thestatus. |

2.Table **tbl_doctor**

Description : The table is used to store doctordetails.

Table 3.2 tbl_doctor

| **Field Name** | **Data Type** | **Constraint** | **Description of Field** |
|---|---|---|---|
| Doctor_id | int | Primary Key | The field is used to uniquely identify Doctor,and the values are automatically generated. |
| Login_id | int | Foreign Key | This field refers the Login Id |
| name | Varchar(30) | Not Null | This field stores the Doctor name |
| email | Varchar(50) | Not Null | This field stores the Doctor email |
| password | Varchar(30) | Not Null | This field stores the Doctor password |
| specialization | Varchar(30) | Not Null | This field stores the Doctor Specialization |
| experience | int | Not Null | This field stores the Doctor Experience |

| | | | |
|---|---|---|---|
| Contact_no | Big Int | Not Null | This field stores the Doctor Contact No |
| Hospital_timimg | time | Not Null | This field stores the Doctor's Hospital Timing |
| status | Varchar(30) | Not Null | This field stores the Doctor's Status(Active/Inactive) |
| Created_at | Date | Not Null | This field stores the Created Date |
| Image | Varchar(300) | Not Null | This field stores the Doctor's Image |

3.Table **tbl_patient**

Description : The table is used to store patient details.

Table 3.3 tbl_patient

| **Field Name** | **Data Type** | **Constraint** | **Description of Field** |
|---|---|---|---|
| Patient_id | int | Primary Key | The field is used to uniquely identify Patient,and the values are automatically generated. |
| Login_id | int | Foreign key | This field refers the Login Id |
| Doctor_id | int | Foreign Key | This field refers the Doctor Id |
| name | Varchar(20) | Not Null | This field refers the Patient Name |
| email | Varchar(60) | Not Null | This field refers the Patient email |
| password | Varchar(20) | Not Null | This field refers the Patient password |

| | | | |
|---|---|---|---|
| age | int | Not Null | This field refers the Patient age |
| phone | Big Int | Not Null | This field refers the Patient phone no |
| address | Varchar(70) | Not Null | This field refers the Patient Address |
| Blood_group | Varchar(20) | Not Null | This field refers the Patient Blood Group |
| Last_cycle_date | Date | Not Null | This field refers the Patient last cycle date |
| Current_trimester | int | Not Null | This field refers the Patient Current trimester |
| Edd_date | Date | Not Null | This field refers the Patient Expected Delivery Date |
| Delivery_date | Date | Not Null | This field refers the Patient Delivery Date |
| Profile_lock_status | Varchar(20) | Not Null | This field refers the Patient profile lock status |
| Lock_start_date | Date | Not Null | This field refers the Patient lock start date |
| status | Varchar(20) | Not Null | This field refers the Patient status(Active/Inactive) |
| Created_at | Date | Not Null | This field refers the Created Date |
| Image | Varchar(600) | Not Null | This field refers the Patient Image |

4.Table **tbl_appointment**

Description : The table is used to store appointment details.

Table 3.4 tbl_appointment

| **Field Name** | **Data Type** | **Constraint** | **Description of Field** |
|---|---|---|---|
| Appointment_id | int | Primary key | The field is used to uniquely identify |

| | | | |
|---|---|---|---|
| | | | Appointment ,and the values are automatically generated. |
| Patient_id | int | Foreign Key | This field refers the Patient Id |
| Doctor_id | int | Foreign Key | This field refers the Doctor_id |
| Appointment_date | Date | Not Null | This field refers the Patient Appointment date |
| Appointment_time | Time | Not Null | This field refers the Patient Appointment Time |
| reason | Varchar (70) | Not Null | This field refers the Reason for appointment |
| status | Varchar (70) | Not Null | This field refers the status |
| Rescheduled_date | Date | Not Null | This field refers the Rescheduled date |
| Rescheduled_time | Time | Not Null | This field refers the Rescheduled time |
| Created_at | Date | Not Null | This field refers the created date |

5.Table **tbl_trimester_update**

Description : The table is used to store trimester details.

Table 3.5 tbl_trimester

| **Field Name** | **Data Type** | **Constraint** | **Description of Field** |
|---|---|---|---|
| Trimester_id | int | Primary Key | The field is used to uniquely identify |

| | | | |
|---|---|---|---|
| | | | Patient Trimester ,and the values are automatically generated. |
| Doctor_id | int | Foreign Key | This field refers the Doctor Id |
| Patient_id | int | Foreign Key | This field refers the Patient Id |
| Trimester_number | int | Not Null | This field refers the Trimester Number |
| title | Varchar(30) | Not Null | This field refers the Title of Trimester |
| description | Varchar(80) | Not Null | This field refers the Description of Trimester |
| Create_at | Date | Not Null | This field refers the Creation Date |

6.Table **tbl_prescription**

Description : The table is used to store prescription details.

Table 3.6 tbl_prescription

| **Field Name** | **Data Type** | **Constraint** | **Description of Field** |
|---|---|---|---|
| Prescription_id | Int | Primary Key | The field is used to uniquely identify Patient Prescription from doctor ,and the values are automatically generated. |
| Doctor_id | Int | Foreign Key | This field refers the Doctor Id |
| Patient_id | Int | Foreign Key | This field refers the Patient Id |

| | | | |
|---|---|---|---|
| Appointment_id | Int | Foreign Key | This field refers the Appointment Id |
| Prescription_file | Varchar(600) | Not Null | This field refers the Prescription File |
| notes | Varchar(80) | Not Null | This field refers the Doctor's Notes to the Patient |
| Created_at | Date | Not Null | This field refers the Creation Date |

7.Table **tbl_medical_record**

Description : The table is used to store medical record details.

Table 3.7 tbl_medical_record

| **Field Name** | **Data Type** | **Constraint** | **Description of Field** |
|---|---|---|---|
| Record_id | Int | Primary Key | The field is used to uniquely identify Patient Record ,and the values are automatically generated. |
| Patient_id | Int | Foreign Key | This field refers the Patient Id |

| | | | |
|---|---|---|---|
| Uploaded_by | Varchar(30) | Not Null | This field refers Who uploaded the file |
| Patient_uploaded_file | Varchar(600) | Not Null | This field refers the patient uploaded file of vitals which have been done in External Labs. |
| Admin_uploaded_file | Varchar(600) | Not Null | This field refers the Admin uploaded file which have done from the Hospital Itself |
| Doctor_view_status | Varchar(30) | Not Null | This field refers the Doctor View Status |
| description | Varchar(100) | Not Null | This field refers the Description |
| Record_date | Date | Not Null | This field refers the Record Date |
| Created_at | Date | Not Null | This field refers the Creation Date |

8.Table **tbl_visit_history**

Description : The table is used to store patient visit history details.

Table 3.8 tbl_visit_history

| **Field Name** | **Data Type** | **Constraint** | **Description of Field** |
|---|---|---|---|
| Visit_id | Int | Primary Key | The field is used to uniquely identify Patient Visit ,and the values are automatically generated. |

| | | | |
|---|---|---|---|
| Patient_id | Int | Foreign Key | This field refers the Patient Id |
| Doctor_id | Int | Foreign Key | This field refers the Doctor Id |
| Appointment_id | Int | Foreign Key | This field refers the Appointment Id |
| Visit_date | Date | Not Null | This field refers the Visit Date |
| Details | Varchar(100) | Not Null | This field refers the Details |
| weight | Float | Not Null | This field refers the Weight |
| Blood_pressure | Varchar(30) | Not Null | This field refers the Blood Pressure |
| Health_status | Varchar(30) | Not Null | This field refers the Health Status of the Patient |
| Next_visit_date | Date | Not Null | This field refers the Next Visit Date |
| Created_at | Date | Not Null | This field refers the Creation Date |

9.Table **tbl_notifications**

Description : The table is used to store notification details

Table 3.9 tbl_notifications

| **Field Name** | **Data Type** | **Constraint** | **Description of Field** |
|---|---|---|---|
| Notification_id | Int | Primary Key | The field is used to uniquely identify Notifications ,and the values are automatically generated. |
| User_id | Int | Foreign Key | This field refers the User Id |

| | | | |
|---|---|---|---|
| User_role | Varchar(30) | Not Null | This field refers the User role(Admin/Patient/Doctor) |
| title | Varchar(30) | Not Null | This field refers the Title of Notification |
| message | Varchar(100) | Not Null | This field refers the Message to be delivered to the receiver. |
| status | Varchar(30) | Not Null | This field refers the Status(Read/Unread) |
| Created_at | Date | Not Null | This field refers the Creation Date |
| notification_type | Varchar(30) | Not Null | This field refers the Notification Type. |
| Reference_id | Date | Not Null | This field refers the Reference Id |
| Reference_type | Varchar(30) | Not Null | This field refers the Reference Type |

10.Table **tbl_delivery_details**

Description : The table is used to store delivery details.

Table 3.10 tbl_delivery_details

| **Field Name** | **Data Type** | **Constraint** | **Description of Field** |
|---|---|---|---|
| Delivery_id | Int | Primary Key | The field is used to uniquely identify Patient Delivery ,and the values are automatically generated. |
| Patient_id | Int | Foreign Key | This field refers the Patient Id |
| Doctor_id | Int | Foreign Key | This field refers the Doctor Id |
| Delivery_date | Date | Not Null | This field refers the Delivery Date |

| | | | |
|---|---|---|---|
| Delivery_type | Varchar(100) | Not Null | This field refers the Delivery Type |
| Baby_weight | Float | Not Null | This field refers the Baby weight |
| Mother_condition | Varchar(100) | Not Null | This field refers the Condition of the Mother |
| remarks | Varchar(100) | Not Null | This field refers the Remarks |
| Created_at | Date | Not Null | This field refers the Creation Date |

11.Table **tbl_doctor_leave**

Description : The table is used to store doctor leave details.

Table 3.11 tbl_doctor_leave

| **Field Name** | **Data Type** | **Constraint** | **Description of Field** |
|---|---|---|---|
| Leave_id | Int | Primary Key | The field is used to uniquely identify Doctor's Leave ,and the values are automatically generated. |
| Doctor_id | Int | Primary Key | This field refers the Doctor Id |
| Leave_date | Date | Not Null | This field refers the Leave Date |
| reason | Varchar(100) | Not Null | This field refers the leave Reason |